FinanceApp

Обзор программного кода

Информационная система для учёта личных финансов

C# / .NET 8 / WPF / Entity Framework Core / SQLite

---

1 Общее описание приложения

FinanceApp — это десктопное приложение для операционной системы Windows, предназначенное для учёта личных финансов. Приложение позволяет пользователям:

- вести учёт **доходов** (зарплата, фриланс, кэшбэк и др.);
- вести учёт **расходов** (продукты, транспорт, аренда и др.);
- ставить **финансовые цели** и отслеживать прогресс их достижения;
- видеть **сводку** по балансу (общий доход, расход, разница);
- **экспортировать** финансовый отчёт в документ Word (.docx);
- **администрировать** пользователей (для роли администратора).

Приложение написано на языке **C#** с использованием платформы **.NET 8** и фреймворка **WPF** (Windows Presentation Foundation) для создания графического интерфейса. Данные хранятся в файловой базе данных **SQLite**, которая не требует установки отдельного сервера — база создаётся автоматически при первом запуске.

2 Архитектура приложения

Код приложения разделён на 4 логических слоя (папки), каждый из которых отвечает за свою область:

Папка	Что находится и зачем
Models/	**Модели данных** — 5 классов, описывающих структуру таблиц в базе данных. Каждый класс соответствует одной таблице: пользователь, расход, доход, цель, связь пользователя с целью.
Data/	**Работа с базой данных** — класс FinanceDbContext, который управляет подключением к SQLite, описывает связи между таблицами и содержит начальные данные (seed).
Windows/	**Окна приложения** — 4 окна с графическим интерфейсом: авторизация, регистрация, главное окно пользователя (5 вкладок), панель администратора. Каждое окно состоит из двух файлов: .xaml (внешний вид) и .xaml.cs (логика поведения).
Helpers/	**Вспомогательные классы** — хеширование паролей (BCryptHelper) и экспорт данных в Word (WordExporter).

Такое разделение позволяет легко находить нужный код: если нужно изменить внешний вид — идём в Windows/, если нужно поменять структуру данных — идём в Models/.

3 Структура файлов проекта

Ниже показано дерево всех файлов проекта с пояснением назначения каждого:

```
FinanceApp/
│
├── FinanceApp.sln                  — файл решения Visual Studio
│                                     (открывается двойным кликом)
│
└── FinanceApp/                     — основная папка проекта
    │
    ├── FinanceApp.csproj           — настройки проекта: целевая
    │                                 платформа (.NET 8), список
    │                                 подключённых библиотек
    │
    ├── App.xaml                    — точка входа: указывает, какое
    │                                 окно открывается первым
    ├── App.xaml.cs                 — (авто) код запуска приложения
    │
    ├── Models/                     — МОДЕЛИ ДАННЫХ (таблицы БД)
    │   ├── User.cs                 — пользователь: логин, пароль,
    │   │                             фото, роль (юзер/админ)
    │   ├── Expense.cs              — расход: название, сумма, дата
    │   ├── Income.cs               — доход: название, сумма, дата
    │   ├── Target.cs               — цель: название, сумма, сроки
    │   └── UserTarget.cs           — связь: какая цель у какого
    │                                 пользователя
    │
    ├── Data/                       — БАЗА ДАННЫХ
    │   └── FinanceDbContext.cs      — подключение к SQLite,
    │                                 описание связей между
    │                                 таблицами, начальные данные
    │
    ├── Windows/                    — ОКНА ПРИЛОЖЕНИЯ
    │   ├── LoginWindow.xaml        — внешний вид окна входа
    │   ├── LoginWindow.xaml.cs     — логика: проверка пароля,
    │   │                             переход в нужное окно
    │   ├── RegisterWindow.xaml     — внешний вид окна регистрации
    │   ├── RegisterWindow.xaml.cs  — логика: проверка полей,
    │   │                             создание пользователя
    │   ├── MainWindow.xaml         — внешний вид главного окна
    │   │                             (5 вкладок)
    │   ├── MainWindow.xaml.cs      — логика: CRUD расходов/доходов,
    │   │                             цели, экспорт, сводка
    │   ├── AdminWindow.xaml        — внешний вид панели админа
    │   └── AdminWindow.xaml.cs     — логика: список пользователей,
    │                                 удаление, статистика
    │
    └── Helpers/                    — ВСПОМОГАТЕЛЬНЫЕ КЛАССЫ
        ├── BCryptHelper.cs         — хеширование паролей SHA-256
        └── WordExporter.cs         — генерация отчёта в .docx
```

4 База данных

4.1 Общая схема

Приложение использует **SQLite** — файловую базу данных. Файл `FinanceDB.db` создаётся автоматически при первом запуске рядом с исполняемым файлом. Никаких серверов устанавливать не нужно.

В базе 5 таблиц:

Таблица	Что хранит	Связана с
Users	Пользователи: логин, хеш пароля, фото, роль (0=пользователь, 1=администратор)	Expenses, Incomes, UserTargets
Expenses	Расходы: название расхода, сумма в рублях, дата	Users (каждый расход принадлежит одному пользователю)
Incomes	Доходы: название дохода, сумма в рублях, дата	Users (каждый доход принадлежит одному пользователю)
Targets	Финансовые цели: название, целевая сумма, дата начала и окончания	UserTargets
UserTargets	Связующая таблица: какому пользователю принадлежит какая цель	Users и Targets

4.2 Подробное описание полей каждой таблицы

Таблица Users (Пользователи):

Поле	Тип	Описание
UserID	INTEGER	Уникальный номер пользователя (первичный ключ, присваивается автоматически)
Username	TEXT(50)	Логин пользователя — уникальный, до 50 символов
PasswordHash	TEXT(255)	Хеш пароля (SHA-256) — сам пароль НЕ хранится в базе
Photo	TEXT	Путь к фотографии пользователя (необязательное поле)
Role	INTEGER	Роль: 0 = обычный пользователь, 1 = администратор

Таблица Expenses (Расходы):

Поле	Тип	Описание
ExpenseID	INTEGER	Уникальный номер расхода (первичный ключ)
UserID	INTEGER	Ссылка на пользователя, которому принадлежит расход (внешний ключ)
ExpenseName	TEXT(100)	Название расхода, например: «Продукты», «Транспорт»
ExpenseAmount	REAL	Сумма расхода в рублях, например: 3500.00
ExpenseDate	DATETIME	Дата совершения расхода

Таблица Incomes (Доходы):

Поле	Тип	Описание
IncomeID	INTEGER	Уникальный номер дохода (первичный ключ)
UserID	INTEGER	Ссылка на пользователя (внешний ключ)
IncomeName	TEXT(100)	Название дохода, например: «Зарплата», «Фриланс»
IncomeAmount	REAL	Сумма дохода в рублях, например: 55000.00
IncomeDate	DATETIME	Дата получения дохода

Таблица Targets (Финансовые цели):

Поле	Тип	Описание
TargetID	INTEGER	Уникальный номер цели (первичный ключ)
TargetName	TEXT(100)	Название цели, например: «Отпуск», «Новый ноутбук»
TargetAmount	REAL	Целевая сумма в рублях, которую нужно накопить
StartDate	DATETIME	Дата начала накопления
EndDate	DATETIME	Дата, к которой нужно достичь цели

Таблица UserTargets (Связь пользователей и целей):

Поле	Тип	Описание
UserTargetsID	INTEGER	Уникальный номер связи (первичный ключ)
UserID	INTEGER	Ссылка на пользователя (внешний ключ → Users)
TargetID	INTEGER	Ссылка на цель (внешний ключ → Targets)

5 Модели данных (Models/)

Модели — это классы C#, которые описывают структуру каждой таблицы в базе данных. Entity Framework Core использует эти классы для автоматического создания таблиц.

5.1 User.cs — Пользователь

```
// Указываем, что этот класс соответствует таблице "Users"
[Table("Users")]
public class User
{
    [Key]  // Первичный ключ — уникальный номер
    public int UserID { get; set; }

    [Required]           // Обязательное поле
    [MaxLength(50)]      // Максимум 50 символов
    public string Username { get; set; } = string.Empty;

    [Required]
    [MaxLength(255)]
    public string PasswordHash { get; set; } = string.Empty;

    public string? Photo { get; set; }  // ? означает необязательное

    // Роль: 0 = обычный пользователь, 1 = администратор
    public int Role { get; set; } = 0;

    // Навигационные свойства — связи с другими таблицами:
    // У одного пользователя может быть много расходов
    public ICollection<Expense> Expenses { get; set; }
        = new List<Expense>();
    // У одного пользователя может быть много доходов
    public ICollection<Income> Incomes { get; set; }
        = new List<Income>();
    // У одного пользователя может быть много целей
    public ICollection<UserTarget> UserTargets { get; set; }
        = new List<UserTarget>();
}
```

Пояснение: `[Table("Users")]` — атрибут, который говорит Entity Framework создать таблицу с именем «Users». `[Key]` — помечает поле как первичный ключ (уникальный идентификатор каждой записи). `[Required]` — поле обязательное, не может быть пустым. `ICollection<Expense>` — означает, что у одного пользователя может быть список (коллекция) расходов.

5.2 Expense.cs — Расход

```
[Table("Expenses")]
public class Expense
{
    [Key]
    public int ExpenseID { get; set; }

    [ForeignKey("User")]  // Внешний ключ — связь с таблицей Users
    public int UserID { get; set; }
```

```
    [Required]
    [MaxLength(100)]
    public string ExpenseName { get; set; } = string.Empty;

    // Тип decimal(18,2) — числа с двумя знаками после запятой
    // Подходит для денежных сумм: 3500.00, 1200.50 и т.д.
    [Column(TypeName = "decimal(18,2)")]
    public decimal ExpenseAmount { get; set; }

    // Дата расхода, по умолчанию — текущая
    public DateTime ExpenseDate { get; set; } = DateTime.Now;

    // Навигационное свойство — ссылка на пользователя
    public User? User { get; set; }
}
```

Пояснение: `[ForeignKey("User")]` — внешний ключ, связывающий расход с конкретным пользователем. Это означает, что каждый расход «принадлежит» одному пользователю, и система знает, какому именно.

5.3 Income.cs — Доход

```
[Table("Incomes")]
public class Income
{
    [Key]
    public int IncomeID { get; set; }

    [ForeignKey("User")]
    public int UserID { get; set; }

    [Required]
    [MaxLength(100)]
    public string IncomeName { get; set; } = string.Empty;

    [Column(TypeName = "decimal(18,2)")]
    public decimal IncomeAmount { get; set; }

    public DateTime IncomeDate { get; set; } = DateTime.Now;

    public User? User { get; set; }
}
```

Структура полностью аналогична расходам, но хранит информацию о доходах.

5.4 Target.cs — Финансовая цель

```
[Table("Targets")]
public class Target
{
    [Key]
    public int TargetID { get; set; }

    [Required]
    [MaxLength(100)]
    public string TargetName { get; set; } = string.Empty;
```

```
    [Column(TypeName = "decimal(18,2)")]
    public decimal TargetAmount { get; set; }  // Сколько нужно накопить

    public DateTime StartDate { get; set; } = DateTime.Now;
    public DateTime EndDate { get; set; }      // К какому числу

    public ICollection<UserTarget> UserTargets { get; set; }
        = new List<UserTarget>();
}
```

5.5 UserTarget.cs — Связь пользователя и цели

```
[Table("UserTargets")]
public class UserTarget
{
    [Key]
    public int UserTargetsID { get; set; }

    [ForeignKey("User")]
    public int UserID { get; set; }     // Какой пользователь

    [ForeignKey("Target")]
    public int TargetID { get; set; }   // Какая цель

    public User? User { get; set; }
    public Target? Target { get; set; }
}
```

Пояснение: эта таблица нужна для реализации связи «многие ко многим» — один пользователь может иметь несколько целей, и одна цель теоретически может принадлежать нескольким пользователям.

6 Контекст базы данных (Data/FinanceDbContext.cs)

Контекст — это главный класс для работы с базой данных. Он указывает, какие таблицы существуют, как они связаны, и содержит начальные данные.

```
public class FinanceDbContext : DbContext
{
    // Каждое свойство DbSet<T> = одна таблица в базе данных
    public DbSet<User> Users { get; set; }
    public DbSet<Expense> Expenses { get; set; }
    public DbSet<Income> Incomes { get; set; }
    public DbSet<Target> Targets { get; set; }
    public DbSet<UserTarget> UserTargets { get; set; }

    // Настройка подключения — SQLite, файл FinanceDB.db
    protected override void OnConfiguring(
        DbContextOptionsBuilder optionsBuilder)
    {
        var dbPath = Path.Combine(
            AppDomain.CurrentDomain.BaseDirectory,
            "FinanceDB.db");
        optionsBuilder.UseSqlite($"Data Source={dbPath}");
    }

    // Описание связей между таблицами и начальные данные
    protected override void OnModelCreating(
        ModelBuilder modelBuilder)
    {
        // Связь: один пользователь → много расходов
        modelBuilder.Entity<Expense>()
            .HasOne(e => e.User)
            .WithMany(u => u.Expenses)
            .HasForeignKey(e => e.UserID);

        // Связь: один пользователь → много доходов
        modelBuilder.Entity<Income>()
            .HasOne(i => i.User)
            .WithMany(u => u.Incomes)
            .HasForeignKey(i => i.UserID);

        // Связь: пользователь ↔ цели (через UserTargets)
        modelBuilder.Entity<UserTarget>()
            .HasOne(ut => ut.User)
            .WithMany(u => u.UserTargets)
            .HasForeignKey(ut => ut.UserID);

        modelBuilder.Entity<UserTarget>()
            .HasOne(ut => ut.Target)
            .WithMany(t => t.UserTargets)
            .HasForeignKey(ut => ut.TargetID);

        // Начальные данные — создаются автоматически
        // Администратор: admin / admin123
        modelBuilder.Entity<User>().HasData(new User
```

```
        {
            UserID = 1,
            Username = "admin",
            PasswordHash = BCryptHelper.HashPassword("admin123"),
            Role = 1  // администратор
        });

        // Пользователь: 123qwe / 1234
        modelBuilder.Entity<User>().HasData(new User
        {
            UserID = 2,
            Username = "123qwe",
            PasswordHash = BCryptHelper.HashPassword("1234"),
            Role = 0  // обычный пользователь
        });

        // Демо-расходы для пользователя 123qwe
        modelBuilder.Entity<Expense>().HasData(
            new Expense { ExpenseID = 1, UserID = 2,
                ExpenseName = "Продукты",
                ExpenseAmount = 3500.00m,
                ExpenseDate = new DateTime(2025, 3, 1) },
            new Expense { ExpenseID = 2, UserID = 2,
                ExpenseName = "Транспорт",
                ExpenseAmount = 1200.00m,
                ExpenseDate = new DateTime(2025, 3, 3) },
            // ... ещё 3 расхода
        );

        // Демо-доходы для пользователя 123qwe
        modelBuilder.Entity<Income>().HasData(
            new Income { IncomeID = 1, UserID = 2,
                IncomeName = "Зарплата",
                IncomeAmount = 55000.00m,
                IncomeDate = new DateTime(2025, 3, 1) },
            // ... ещё 2 дохода
        );

        // Демо-цели
        modelBuilder.Entity<Target>().HasData(
            new Target { TargetID = 1,
                TargetName = "Отпуск",
                TargetAmount = 80000.00m,
                StartDate = new DateTime(2025, 1, 1),
                EndDate = new DateTime(2025, 7, 1) },
            // ... ещё 1 цель
        );
    }
}
```

7 Хеширование паролей (Helpers/BCryptHelper.cs)

Пароли **никогда не хранятся в открытом виде**. Вместо этого пароль преобразуется в «хеш» — необратимую строку символов. При входе система хеширует введённый пароль и сравнивает с сохранённым хешем.

```
public static class BCryptHelper
{
    // Преобразование пароля в хеш
    public static string HashPassword(string password)
    {
        // Создаём экземпляр алгоритма SHA-256
        using var sha256 = SHA256.Create();

        // Преобразуем пароль в байты и вычисляем хеш
        var bytes = sha256.ComputeHash(
            Encoding.UTF8.GetBytes(password));

        // Преобразуем байты хеша в строку Base64
        return Convert.ToBase64String(bytes);
    }

    // Проверка пароля: хешируем введённый и сравниваем
    public static bool VerifyPassword(
        string password, string hash)
    {
        var passwordHash = HashPassword(password);
        return passwordHash == hash;
        // true = пароль верный, false = неверный
    }
}
```

Пример: пароль `"1234"` ⟶ хеш `"A6xnQhbz4Vx2HuGl..."` (необратимо).

8 Экспорт в Word (Helpers/WordExporter.cs)

Экспорт реализован с помощью библиотеки **DocumentFormat.OpenXml**, которая создаёт файлы .docx без необходимости установки Microsoft Office.

```
public static class WordExporter
{
    public static void Export(string filePath,
        string username,
        List<Income> incomes,
        List<Expense> expenses)
    {
        // Создаём новый документ Word
        using var doc = WordprocessingDocument.Create(
            filePath,
            WordprocessingDocumentType.Document);

        var mainPart = doc.AddMainDocumentPart();
        mainPart.Document = new Document();
        var body = mainPart.Document
            .AppendChild(new Body());
```

```
        // Заголовок: "Финансовый отчёт — username"
        AddParagraph(body,
            $"Финансовый отчёт — {username}",
            bold: true, fontSize: "28");

        // Дата формирования
        AddParagraph(body,
            $"Дата: {DateTime.Now:dd.MM.yyyy HH:mm}",
            bold: false, fontSize: "20");

        // Сводка: общий доход, расход, баланс
        var totalIncome = incomes.Sum(i => i.IncomeAmount);
        var totalExpense = expenses.Sum(e => e.ExpenseAmount);
        var balance = totalIncome - totalExpense;

        AddParagraph(body,
            $"Общий доход: {totalIncome:N2} ₽",
            bold: true, fontSize: "24");
        AddParagraph(body,
            $"Общий расход: {totalExpense:N2} ₽",
            bold: true, fontSize: "24");
        AddParagraph(body,
            $"Баланс: {balance:N2} ₽",
            bold: true, fontSize: "24");

        // Таблица доходов (с рамками, Times New Roman)
        var incomeTable = CreateTable(body);
        AddTableRow(incomeTable,
            new[] { "Название", "Сумма", "Дата" },
            isHeader: true);
        foreach (var inc in incomes)
        {
            AddTableRow(incomeTable, new[]
            {
                inc.IncomeName,
                $"{inc.IncomeAmount:N2} ₽",
                inc.IncomeDate.ToString("dd.MM.yyyy")
            }, isHeader: false);
        }

        // Аналогичная таблица расходов
        // ...

        mainPart.Document.Save();
    }
}
```

Результат экспорта: документ .docx с заголовком, сводкой и двумя таблицами (доходы и расходы), оформленный шрифтом Times New Roman.

9 Окна приложения (Windows/)

9.1 LoginWindow — Окно авторизации

Первое окно, которое видит пользователь. Содержит два поля ввода (логин и пароль) и две кнопки (Войти и Регистрация).

Логика входа (LoginWindow.xaml.cs):

```
private void LoginButton_Click(object sender, RoutedEventArgs e)
{
    var username = UsernameTextBox.Text.Trim();
    var password = PasswordBox.Password;

    // Проверяем, что поля не пустые
    if (string.IsNullOrEmpty(username)
        || string.IsNullOrEmpty(password))
    {
        MessageBox.Show("Заполните все поля!");
        return;
    }

    // Подключаемся к базе данных
    using var db = new FinanceDbContext();
    db.Database.EnsureCreated();  // Создать БД, если не существует

    // Ищем пользователя по логину
    var user = db.Users
        .FirstOrDefault(u => u.Username == username);

    // Проверяем пароль
    if (user == null
        || !BCryptHelper.VerifyPassword(password, user.PasswordHash))
    {
        MessageBox.Show("Неправильный логин или пароль!");
        return;
    }

    MessageBox.Show("Авторизация прошла успешно!");

    // Открываем окно в зависимости от роли
    if (user.Role == 1)  // Администратор
    {
        var adminWindow = new AdminWindow(user);
        adminWindow.Show();
    }
    else  // Обычный пользователь
    {
        var mainWindow = new MainWindow(user);
        mainWindow.Show();
    }

    Close();  // Закрываем окно входа
}
```

9.2 RegisterWindow — Окно регистрации

Содержит три поля: логин, пароль, подтверждение пароля. Проверяет:

- все поля заполнены;
- пароли совпадают;
- пароль не короче 4 символов;
- логин уникален (не занят другим пользователем).

```
private void RegisterButton_Click(object sender, RoutedEventArgs e)
{
    // ... проверки полей ...

    using var db = new FinanceDbContext();

    // Проверяем, не занят ли логин
    if (db.Users.Any(u => u.Username == username))
    {
        MessageBox.Show("Пользователь уже существует!");
        return;
    }

    // Создаём нового пользователя с хешированным паролем
    var user = new User
    {
        Username = username,
        PasswordHash = BCryptHelper.HashPassword(password),
        Role = 0  // обычный пользователь
    };

    db.Users.Add(user);     // Добавляем в базу
    db.SaveChanges();       // Сохраняем изменения

    MessageBox.Show("Регистрация прошла успешно!");
    Close();
}
```

9.3 MainWindow — Главное окно (5 вкладок)

Самое большое окно приложения. Содержит:

1. **Общие данные** — сводка по финансам (доход, расход, баланс) + таблица последних транзакций
2. **Расходы** — таблица расходов + форма добавления + кнопка удаления
3. **Доходы** — аналогично расходам
4. **Финансовые цели** — карточки целей с прогресс-барами
5. **Экспорт** — кнопка для выгрузки в Word

Загрузка сводки (вкладка «Общие данные»):

```
private void LoadDashboard()
{
    using var db = new FinanceDbContext();

    // Загружаем все доходы и расходы текущего пользователя
    var incomes = db.Incomes
        .Where(i => i.UserID == _currentUser.UserID)
        .ToList();
    var expenses = db.Expenses
        .Where(e => e.UserID == _currentUser.UserID)
        .ToList();

    // Считаем итоги
    var totalIncome = incomes.Sum(i => i.IncomeAmount);
    var totalExpense = expenses.Sum(e => e.ExpenseAmount);
    var balance = totalIncome - totalExpense;

    // Обновляем текст на экране
    TotalIncomeText.Text = $"{totalIncome:N2} ₽";
    TotalExpenseText.Text = $"{totalExpense:N2} ₽";
    BalanceText.Text = $"{balance:N2} ₽";
}
```

Добавление расхода (вкладка «Расходы»):

```
private void AddExpense_Click(object sender, RoutedEventArgs e)
{
    var name = ExpenseNameBox.Text.Trim();

    // Проверяем, что введено название
    if (string.IsNullOrEmpty(name))
    {
        MessageBox.Show("Введите название расхода!");
        return;
    }

    // Проверяем, что сумма — корректное число
    if (!decimal.TryParse(ExpenseAmountBox.Text,
            out var amount) || amount <= 0)
    {
        MessageBox.Show("Введите корректную сумму!");
        return;
    }
```

```
    var date = ExpenseDatePicker.SelectedDate ?? DateTime.Today;

    // Создаём новую запись в базе данных
    using var db = new FinanceDbContext();
    db.Expenses.Add(new Expense
    {
        UserID = _currentUser.UserID,
        ExpenseName = name,
        ExpenseAmount = amount,
        ExpenseDate = date
    });
    db.SaveChanges();  // Сохраняем в базу

    // Очищаем поля ввода и обновляем таблицу
    ExpenseNameBox.Clear();
    ExpenseAmountBox.Clear();
    LoadExpenses();
}
```

Удаление с подтверждением:

```
private void DeleteExpense_Click(object sender, RoutedEventArgs e)
{
    // Проверяем, что пользователь выбрал строку в таблице
    if (ExpensesGrid.SelectedItem is not Expense selected)
    {
        MessageBox.Show("Выберите расход для удаления!");
        return;
    }

    // Спрашиваем подтверждение
    var result = MessageBox.Show(
        $"Удалить \"{selected.ExpenseName}\"?",
        "Подтверждение",
        MessageBoxButton.YesNo);

    if (result == MessageBoxResult.Yes)
    {
        using var db = new FinanceDbContext();
        var expense = db.Expenses.Find(selected.ExpenseID);
        if (expense != null)
        {
            db.Expenses.Remove(expense);
            db.SaveChanges();
        }
        LoadExpenses();  // Обновляем таблицу
    }
}
```

Финансовые цели — расчёт прогресса:

```
private void LoadTargets()
{
    using var db = new FinanceDbContext();
```

```
    // Получаем цели текущего пользователя
    var userTargets = db.UserTargets
        .Where(ut => ut.UserID == _currentUser.UserID)
        .Include(ut => ut.Target)
        .Select(ut => ut.Target!)
        .ToList();

    // Считаем «сбережения» = доходы минус расходы
    var totalIncome = db.Incomes
        .Where(i => i.UserID == _currentUser.UserID)
        .ToList().Sum(i => i.IncomeAmount);
    var totalExpense = db.Expenses
        .Where(e => e.UserID == _currentUser.UserID)
        .ToList().Sum(e => e.ExpenseAmount);
    var savings = totalIncome - totalExpense;

    // Для каждой цели вычисляем процент прогресса
    var targetViewModels = userTargets.Select(t => new
    {
        t.TargetName,
        t.TargetAmount,
        t.EndDate,
        Progress = t.TargetAmount > 0
            ? Math.Min(100,
                (double)(savings / t.TargetAmount) * 100)
            : 0
        // Пример: сбережения 38720 / цель 80000 = 48.4%
    }).ToList();

    TargetsItemsControl.ItemsSource = targetViewModels;
}
```

9.4 AdminWindow — Панель администратора

Доступна только для пользователей с ролью 1 (администратор). Содержит две вкладки:

1. **Пользователи** — список всех пользователей с возможностью удаления
2. **Статистика** — общие показатели системы

```
private void LoadStats()
{
    using var db = new FinanceDbContext();

    // Количество пользователей
    TotalUsersText.Text = db.Users.Count().ToString();

    // Количество транзакций (доходы + расходы)
    var totalIncomes = db.Incomes.Count();
    var totalExpenses = db.Expenses.Count();
    TotalTransactionsText.Text =
        (totalIncomes + totalExpenses).ToString();

    // Суммы по всей системе
    var allIncomes = db.Incomes.ToList()
        .Sum(i => i.IncomeAmount);
    var allExpenses = db.Expenses.ToList()
        .Sum(e => e.ExpenseAmount);
    AllIncomesText.Text = $"{allIncomes:N2} ₽";
    AllExpensesText.Text = $"{allExpenses:N2} ₽";
}
```

Защита от удаления самого себя:

```
if (userId == _currentUser.UserID)
{
    MessageBox.Show("Нельзя удалить самого себя!");
    return;
}
```

При удалении пользователя система также удаляет все его расходы, доходы и связи с целями — каскадное удаление.

10 Интерфейс (XAML)

Каждое окно описывается на языке XAML — это язык разметки, похожий на HTML. Он определяет, как выглядит окно: какие кнопки, поля ввода, таблицы и тексты расположены на экране.

Пример — кнопка «Войти» в окне авторизации:

```
<Button Content="Войти"
        Height="38" FontSize="14"
        Background="#4CAF50" Foreground="White"
        BorderThickness="0"
        Cursor="Hand"
        Click="LoginButton_Click"/>
```

Пояснение: `Content` — текст на кнопке, `Background` — цвет фона (зелёный), `Click` — какой метод вызвать при нажатии.

11 Технологический стек

Технология	Версия	Для чего используется
C#	12.0	Основной язык программирования — на нём написана вся логика приложения
.NET	8.0	Платформа, на которой работает приложение (аналог «операционной системы» для программы)
WPF	—	Фреймворк для создания десктопных окон, кнопок, таблиц и других элементов интерфейса
Entity Framework Core	8.0.0	ORM — позволяет работать с базой данных через классы C#, без написания SQL-запросов вручную
SQLite	—	Файловая база данных — хранит все данные в одном файле FinanceDB.db, не требует установки сервера
OpenXml SDK	3.0.1	Библиотека для создания документов Word (.docx) без установки Microsoft Office
SHA-256	—	Алгоритм хеширования — превращает пароль в необратимую строку для безопасного хранения

12 NuGet-зависимости

NuGet — это менеджер пакетов для .NET (аналог магазина приложений для библиотек). Проект использует 4 внешних пакета:

```
<!-- Ядро Entity Framework — работа с базой данных -->
<PackageReference
    Include="Microsoft.EntityFrameworkCore"
    Version="8.0.0" />

<!-- Провайдер SQLite — подключение к файловой БД -->
<PackageReference
    Include="Microsoft.EntityFrameworkCore.Sqlite"
    Version="8.0.0" />

<!-- Инструменты EF — миграции и генерация кода -->
<PackageReference
    Include="Microsoft.EntityFrameworkCore.Tools"
    Version="8.0.0" />

<!-- OpenXml — создание документов Word -->
<PackageReference
    Include="DocumentFormat.OpenXml"
    Version="3.0.1" />
```

13 Начальные данные (Seed Data)

При первом запуске приложения в базу данных автоматически добавляются:

Логин	Пароль	Роль	Данные
admin	admin123	Администратор	Только аккаунт, без финансовых данных

123qwe	1234	Пользователь	5 расходов, 3 дохода, 2 финансовые цели

Демо-данные пользователя 123qwe:

- **Расходы:** Продукты (3 500 ₽), Транспорт (1 200 ₽), Аренда квартиры (25 000 ₽), Интернет (600 ₽), Кафе (1 800 ₽)
- **Доходы:** Зарплата (55 000 ₽), Фриланс (15 000 ₽), Кэшбэк (820 ₽)
- **Цели:** Отпуск (80 000 ₽ к 01.07.2025), Новый ноутбук (60 000 ₽ к 01.06.2025)